

LN 13. 动态规划 (Dynamic Programming)

- 核心思想 { 把同一个问题分解成同一类的若干个子问题 (Top-down)
求解最小的子问题. (在求解过程中, 子问题会重复出现. 保存子问题的解)
递归的向上求解. 直到原问题. (Bottom up)
- 可分解问题的性质: 子问题的最优解也是原问题的最优解. (最优性原理)
- * 加权区间调度问题. 例: 暑假课程. 

· 问题定义: 子区间 I_i , 开始时间 s_i , 结束时间 f_i , 权重 $w_i > 0$

要求: 选出相容的 (互不重叠) 区间集合 C

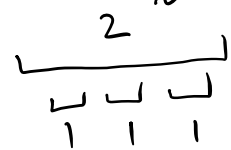
目标: $\max \sum_{i \in C} w_i$

· 贪心是不行的:

hw. 对以下三种, 各举一个反例.

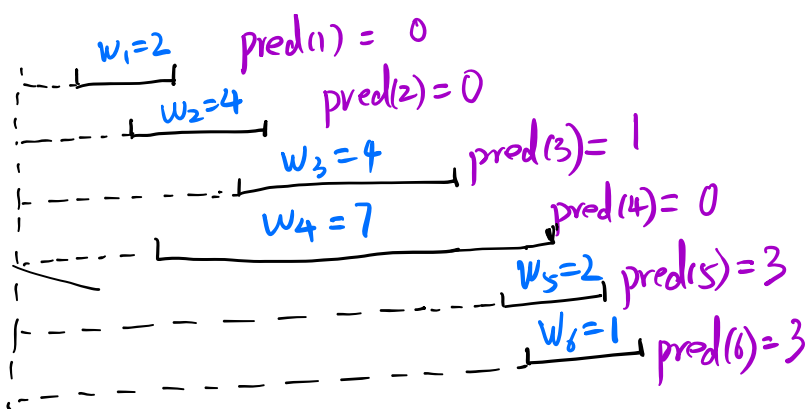
① 不带权 ($w_i = 1$): 最早结束时间的优先.

对于带权, 不行: 

② 按重量优先, 不行 

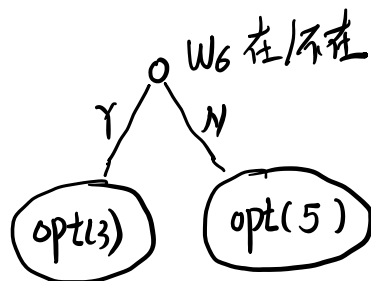
③ 冲突少优先, 不行.  w 均为 1

· 动态规划的设计:



Def. $\text{pred}(i)$: 作为 i 之前驱.
与 i 无重叠之前的最近区间
(若无, 则记为 0)

Def. $\text{opt}(i)$: $\{1, \dots, i\}$ 子序列最大值
(体系按 f_i 从小到大排序)



递归表达式④:

$$\text{opt}(i) = \max(w_i + \text{opt}(\text{pred}(i)), \text{opt}(i-1))$$

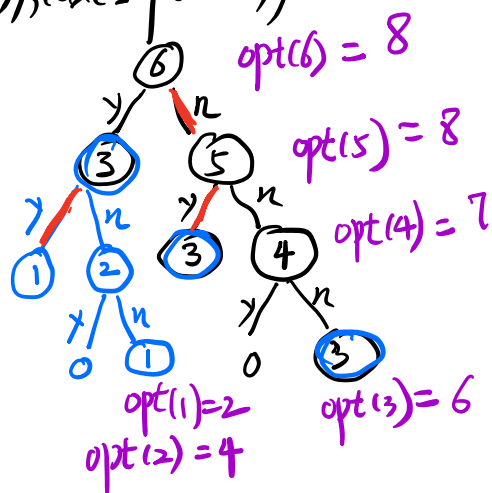
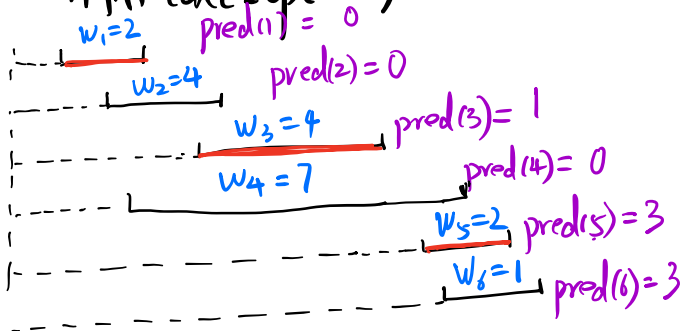
$$\text{opt}(0) = 0$$

递归算法: $\text{calc_opt}(i)$:

if $i=0$, return 0;

else return $\max(w_i + \text{calc_opt}(\text{pred}(i)), \text{calc_opt}(i-1))$

调用 $\text{calc_opt}(n)$



$\text{opt}(6)$ 对应的集合: $\{5, 3, 1\}$

• 时间复杂度: 假设问题已按 f_i 排序, 且存储中间结果. $O(n)$

$opt(0), opt(1), opt(2), \dots, opt(n)$

Q: 若不存储中间结果.

$$T(n) = T(n-1) + T(pred(n))$$

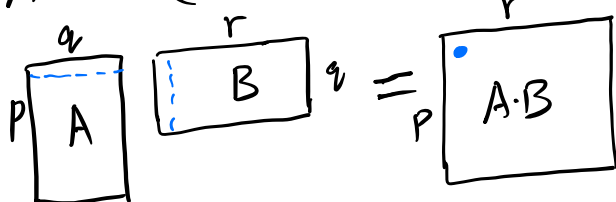
$$\Rightarrow T(n) = T(n-1) + T(n-2) \Rightarrow T(n) \sim \left(\frac{\sqrt{5}+1}{2}\right)^n \approx (1.618)^n$$

斐波拉契数列: 1, 1, 2, 3, 5, 8, 13, 21, ...

* LN 14. 矩阵链乘法.

已知: $A_1, A_2, A_3, \dots, A_n$, 求乘法先后顺序, 使计算复杂度最低.

• 例. $ABC = (AB) \cdot C = A(BC)$



计算复杂度: pqr

$A_{m \times n}, B_{n \times m}, C_{m \times n}$ $(AB)C: 2n^2m$

$A(BC): 2mn^2$

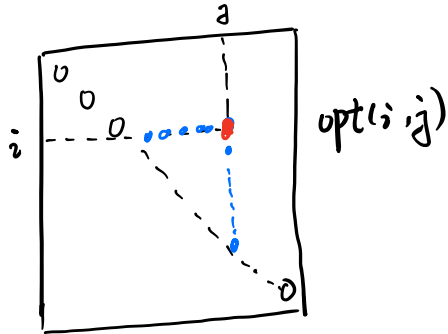
• Def. $C(i, l, j) = \underbrace{(A_i \dots A_l)}_{p \times q} \underbrace{(A_{l+1} \dots A_j)}_{q \times r} = pqr$

• Def. $opt(i, j) = A_i \dots A_j$ 子问题最优计算复杂度

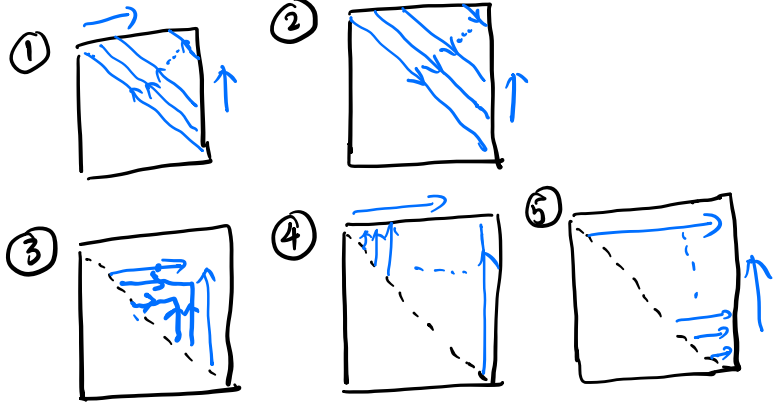
• 递归表达式①: $opt(i, j) = \min_{i \leq l < j} \{ \underbrace{opt(i, l)} + \underbrace{opt(l+1, j)} + C(i, l, j) \}$

$$\text{opt}(i,i) = 0$$

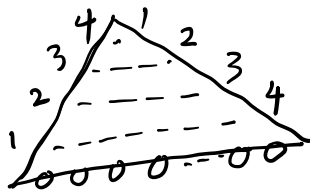
• 保存中间结果:



Q: 计算顺序



算子序列:



• hw: A_1, A_2, A_3, A_4

A_1 : ① 3×6 ② 2×5

A_2 : 6×9 5×10

A_3 : 9×5 10×7

A_4 : 5×4 7×8

	1	2	3	4	
1	0				1
2	X	0			2
3	X	X	0		3
4	X	X	X	0	4

① 生成 $\text{opt}(i,j)$ ② 生成最优的计算顺序

* LN15. 字符串比对.

例. occurrence
| | | | | | | |
occurrence

occurrence
| | | | | | | |
occurrence

Def. δ : 跳过子字符的惩罚

d_{x,y_j} : 等计对应符号第 j 个对应子字符的惩罚: $x_i \sim y_j$

如: $d_{ae}: a \sim e$

• 问题:

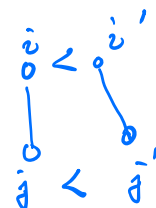
已知两个字符串: $\{x_1, x_2, \dots, x_m\}$

$\{y_1, y_2, \dots, y_n\}$

要求给出一个比对 (alignment): 一个 $\subseteq \Sigma^M$, 且

$\forall (x_i, y_j), (x_{i'}, y_{j'}) \in M$

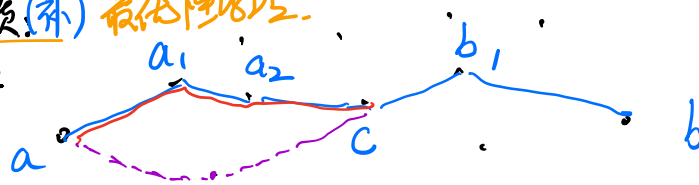
$i < i' \text{ if } j < j'$



目标: 使比对的成本最小.

* DP 的归纳性质 (补) 最优性原理

• 例: 最短路径

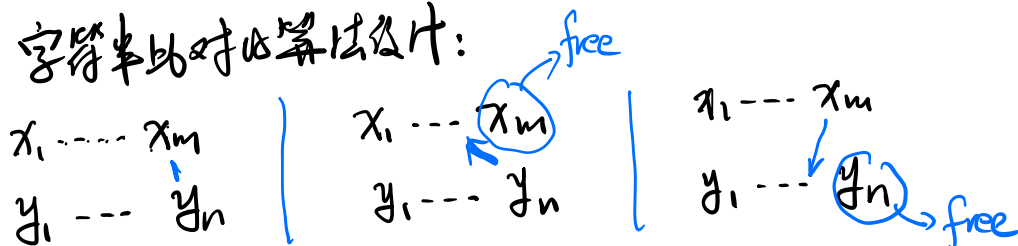


若 $a \sim b$ 的最短路径为 a, a_1, a_2, c, b_1, b .

则 $a \sim c$ 的最短路径为 a, a_1, a_2, c .

(证明: 反证法)

• 字符串比对的算法设计:



• Def. $\text{opt}(i, j)$: $\{x_1 \dots x_i\}$
 $\{y_1 \dots y_j\}$ 子序列的最优匹配成本 (代价总和).

• 递归表达式 (*):
 (状态转移方程) $\text{opt}(i, j) = \min \begin{pmatrix} \text{opt}(i-1, j-1) + d_{x_i, y_j}, \\ \text{opt}(i-1, j) + \delta, \\ \text{opt}(i, j-1) + \delta \end{pmatrix}$

• DP 算法:

1. 初始化:

$$\text{opt}[i, 0] = i \cdot \delta$$

$$\text{opt}[0, j] = j \cdot \delta$$

2. 递归/循环 (对应①)

for $i = 1 \dots m$

for $j = 1 \dots n$

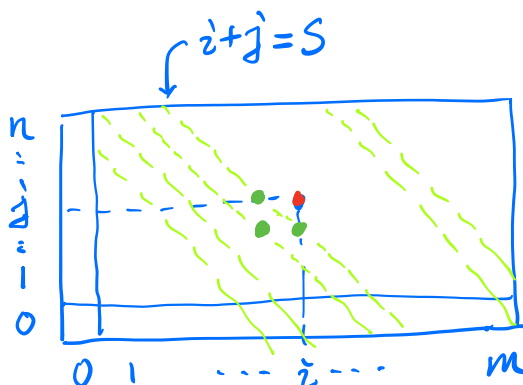
用 (*) 计算 $\text{opt}(i, j)$

end for

end for

3. 返回 $\text{opt}(m, n)$

$x_1 \dots x_i \dots x_m$
 $y_1 \dots y_j \dots y_n$



① $\uparrow\uparrow\uparrow$
 $\rightarrow$ ✓

② $\downarrow\downarrow\downarrow$
 $\rightarrow$ ✗

③ $\rightarrow\rightarrow\rightarrow$
 $\uparrow$ ✓

④ $\rightarrow\downarrow\downarrow$
 $\downarrow$ ✓

⑤ $\rightarrow\uparrow\uparrow$
 $\uparrow$ ✓

• Q: 对应④或⑤的循环:

for $s = 2, \dots, m+n$

for $i = \max(1, s-n)$

to $\min(s-1, m)$

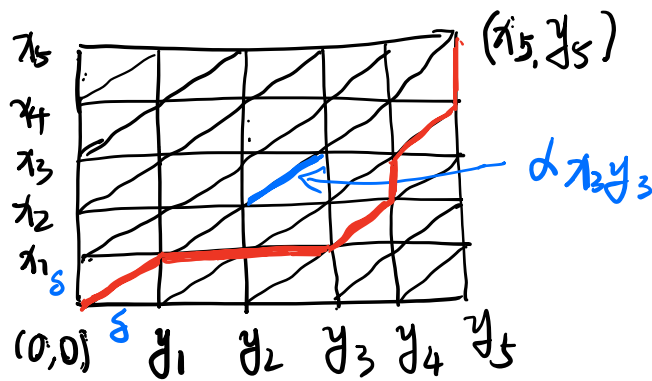
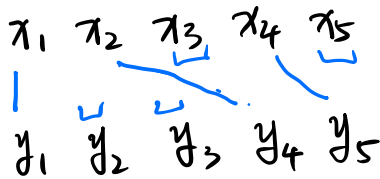
$j = s - i$

计算 $\text{opt}(i, j)$

end for
end for. 对左④

• 计算复杂度: $O(m, n)$

* 设计递归算法:



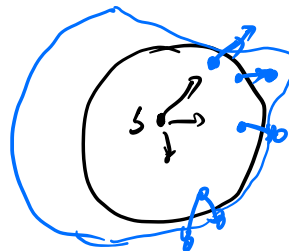
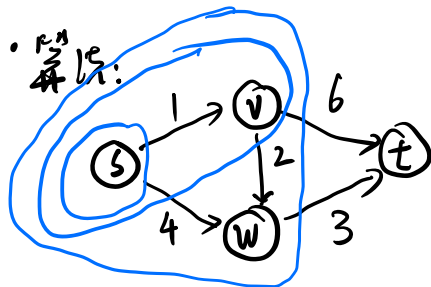
找(0,0)到(5,5)的最短路径

回顾: 最短路径的贪心算法: Dijkstra's 算法

hw. 构造反例
有负边权图.

• 设计: $G=(V, E)$, $w(u, v) \geq 0$.

求: 对 $s \in V$. 计算 $s \rightarrow v \in V$ 的最短路径. $d(s, v)$



X	v	w	t
$\{s\}$	1	4	∞
$\{s, v\}$	1	3	7
$\{s, v, w\}$	1	3	6

算法:

① $s \in X$, $V \setminus X$

② for $|X|=1$ to $|X|=n$

计算 s 到 X 中所有点.

找到 $V \setminus X$ 中的最短路径.

将距离最近的点加入 X

end for.

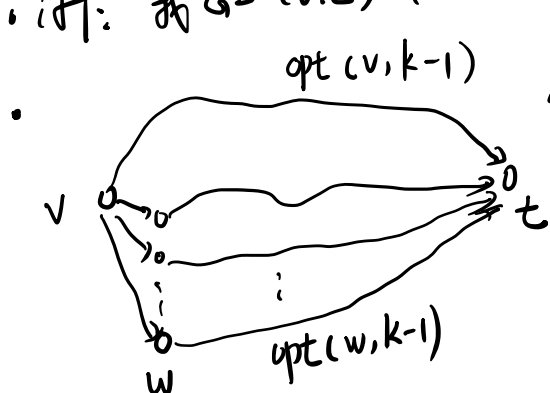
LN.16. 最短路径 (有负边, 无负环): Bellman-ford 算法.

• 引理 (Lemma): 对于 $G=(V, E)$, 可以有负边, 但无负环, 最短路径是一个简单路径.

• Lemma. $|V|=n, |E|=m$. 最短路径最多经过 $n-1$ 条边.

• Def. $\text{opt}(v, k)$: $v \rightsquigarrow t$, 经过 $\leq k$ 条边的最短路径.

• 问: 求 $G=(V, E)$ 中 $s \rightsquigarrow t$ 的最短路径.



$$\textcircled{*}: \text{opt}(v, k) = \min \left(\begin{array}{l} \text{opt}(v, k-1), \\ \min_w \text{opt}(w, k-1) + C_{vw} \end{array} \right)$$

• 输出 $\text{opt}(v, n-1)$

• 边界条件:

$$\text{opt}(v, k) = \begin{cases} 0, & \text{if } v=t \\ \infty, & \text{if } v \neq t, k=0 \end{cases}$$

$\textcircled{*}$

• 算法复杂度:

$$\left. \begin{array}{l} v \in V \\ k: 0 \dots n-1 \end{array} \right\} n \times n \times n, O(n^3)$$

• hw. ① 给出子问题存储的路径结构. 和子问题求解顺序.

② ③ $\rightsquigarrow$ ⑤ $\text{opt}(v, k)$: $s \rightsquigarrow v$, $\leq k$ 条边的最短路径.
给出 ④. 输出和边界条件.

LN17. 0-1 背包问题.

问题定义: n 个物品 $\begin{cases} \text{重量} & w_1, w_2, \dots, w_n \\ \text{价值} & p_1, p_2, \dots, p_n \end{cases}$

背包容量 M .

求: 不超过背包容量, 装入物品的价值最大.

问题形式化定义: 背包 knapsack

$$\begin{aligned} \max & \sum_{i=1}^n p_i x_i \\ \text{s.t.} & \sum_{i=1}^n w_i x_i \leq M \\ & x_i \in \{0, 1\} \\ & p_i, w_i > 0. \end{aligned} \quad \left. \vphantom{\begin{aligned} \max \\ \text{s.t.} \end{aligned}} \right\} \text{记为 } \text{KNAP}(1, n, M)$$

贪心?

例:

w	20	30	40	50	60
p	20	30	44	55	60
p/w	1	1	1.1	1.1	1

$M=100$

贪心策略	x_1	x_2	x_3	x_4	x_5	$\sum x_i p_i$
$w \downarrow$	1	1	1	0	0	94
$p \uparrow$	0	0	1	0	1	104
$p/w \uparrow$	0	0	1	1	0	99
opt	1	1	0	1	0	105

- 0/1 背包问题是 NP 难问题。

穷举: 2^n

- 动态规划 (DP):

$\text{KNAP}(1, j, X)$: $1, 2, \dots, j$, 背包当前还可承重 X

最优子结构性质:

$y_1, y_2, \dots, y_n$ 是 $\text{KNAP}(1, n, M)$ 的 OPT

$y_1 \begin{cases} 1 : \text{若 } \text{KNAP}(2, n, M - w_1) \text{ 的 OPT 是 } y_2, \dots, y_n \\ 0 : \text{若 } \text{KNAP}(2, n, M) \text{ 的 OPT 是 } y_2, \dots, y_n \end{cases}$

- ① 从后向前处理 DP:

1 2 ... $i, \overbrace{i+1, i+2, \dots, n}^{g_{i+1}(x')}, n$
 $\underbrace{\hspace{10em}}_{g_i(x)}$

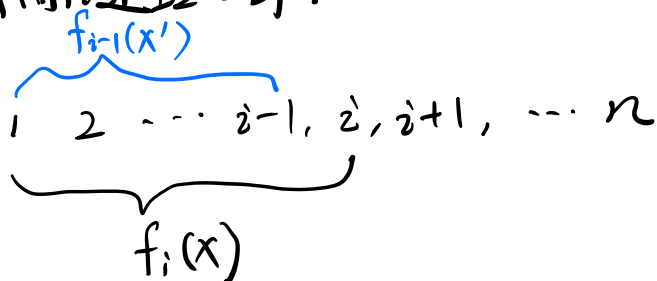
记 $g_i(x)$: $\text{KNAP}(i+1, n, x)$ 的 OPT

$$\textcircled{*} \begin{cases} g_i(x) = \max(g_{i+1}(x), g_{i+1}(x - w_{i+1}) + p_{i+1}) & \text{if } x \geq 0 \\ g_i(x) = -\infty & \text{if } x < 0 \end{cases}$$

初始 (边界条件): $g_n(x) = \begin{cases} 0, & x \geq 0 \\ -\infty, & x < 0 \end{cases}$

结束时返回: $g_0(M)$: $\text{KNAP}(1, n, M)$ 的 OPT.

② 从前向后处理的 DP:



$f_i(x)$: $\text{KNAP}(1, i, x)$ 的 OPT.

hw: 请写出递归公式. 初始条件和递归方程.

LN18. RNA = 二级结构.

DNA: A C G T 脱氧核糖核酸

RNA: A C G U 核糖核酸. $A \sim U, C \sim G$

三叶草
碱基:



· 序列: $x_1 x_2 \dots x_n$

$\text{cost}(i, j)$: $i \sim j$

$d(i)$: i 不匹配

求一个最小成本的匹配.

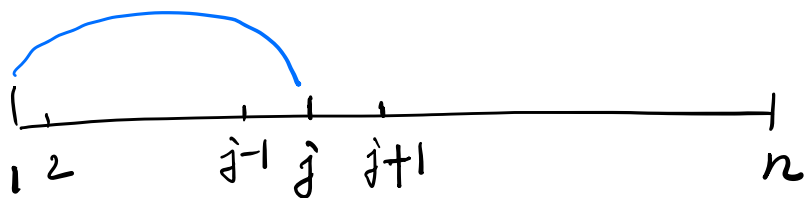
· 生物上的一些事/斜:

① (没书)

② $A \sim U, C \sim G$

③ 无交叉: x

· 动态规划 (DP):



$$\textcircled{*} \text{ MinCost}(1, n) = \min_{j=1 \dots n} \left(C(1, j) + \text{MinCost}(2, j-1) + \text{MinCost}(j+1, n) \right)$$

$$C(i, i) = \alpha(i)$$

$$\text{MinCost}(i, j) = \infty, \quad j < i$$

· 计算复杂度: $\text{MinCost}(1, n)$

$$O(n^3) \begin{cases} O(n^2) & \text{#8177} \\ O(n) & \textcircled{*} \end{cases}$$

LN19. 上下文无关文法及其生成树 (略)